INTERVIEW HANDBOOK

Complete Interview Preparation Guide

ANSIBLE

Automation · Configuration Management · Orchestration

Cloud Operations Engineer → Senior SRE

DevOps Engineer • Platform Engineer • Infrastructure Engineer

150+ Interview Questions	Production Playbooks	Troubleshooting Scenarios
AWS / GCP Integration	Vault & Secrets	CI/CD Pipelines

Interview-First · Production-Focused · AWS-Aligned · SRE Perspective

■ HOW TO USE THIS HANDBOOK

This handbook is designed for engineers with 2–6 years of experience targeting Cloud Operations, DevOps, Platform Engineering, and Senior SRE roles. Every module is interview-weighted — study high-weight modules first. Answers are written to match what interviewers actually want to hear, not textbook theory.

Module	Topic	Priority
1	Ansible Architecture & Fundamentals	■■■■
2	Inventory, Variables & Facts	■■■■■
3	Playbooks & Tasks — Deep Dive	■■■■■
4	Roles, Collections & Galaxy	■■■■■
5	Modules — Essential & Advanced	■■■■■
6	Templating (Jinja2) & Filters	■■■■
7	Ansible Vault & Secrets Management	■■■■■
8	Performance, Optimization & Scaling	■■■■
9	Ansible in CI/CD & AWS	■■■■
10	Production Troubleshooting	■■■■■
11	Top 150 Q&A; + Cheat Sheet	■■■■■

Study Priority

■ Master Modules 2, 3, 4, 5, 7, 10 — these cover ~85% of Ansible questions for SRE/DevOps interviews.
■ For Senior SRE: also master Modules 8, 9 — performance and CI/CD integration are heavily tested.
■ Every ★★★★★ question in Module 11 is near-certain to appear in interviews.

MODULE 1 — Ansible Architecture & Fundamentals

1.1 What is Ansible?

Ansible is an open-source IT automation platform that automates provisioning, configuration management, application deployment, orchestration, and many other IT tasks. Written in Python, it is **agentless** — it communicates with managed nodes over SSH (Linux) or WinRM (Windows) using standard protocols.

Problem (Before Ansible)	Solution (With Ansible)
Manual server config — inconsistent, error-prone	Idempotent playbooks run safely multiple times
Shell scripts not reusable across environments	Roles and collections reusable across projects
Config drift between servers	Every run enforces desired state
Complex orchestration needs custom tooling	Multi-tier deployments with one playbook
Secrets scattered in scripts	Ansible Vault encrypts sensitive data

1.2 Ansible Architecture

Ansible uses a push-based, agentless architecture. The **Control Node** (where Ansible is installed) pushes configuration to **Managed Nodes** via SSH/WinRM. No agents run on managed nodes — only Python must be present (Python 3.x for modern Ansible).

CONTROL NODE (Ansible Installed)	SSH / WinRM → Push-based	MANAGED NODES (No Agent Required)

Component	What It Does	File/Location
Inventory	Defines managed hosts (IPs, groups, variables)	/etc/ansible/hosts or custom
Playbook	YAML file with ordered list of tasks to execute	*.yml (user-defined)
Module	Unit of work (command, file, service, yum, etc.)	Built-in or custom
Role	Reusable, structured collection of tasks/vars/handlers	roles/ directory
Plugin	Extends Ansible (connection, callback, filter, etc.)	Built-in or custom
Vault	Encrypts sensitive data (passwords, keys)	ansible-vault encrypt/decrypt
ansible.cfg	Master configuration file	/etc/ansible/ansible.cfg or local
Galaxy	Community hub for roles and collections	galaxy.ansible.com

1.3 Ansible vs Other Tools

Aspect	Ansible	Chef/Puppet	Terraform	SaltStack
Architecture	Agentless (SSH)	Agent-based	Agentless (API)	Agent (minion)

Language	YAML (Playbooks)	Ruby DSL	HCL	YAML / Python
Approach	Procedural + declarative	Declarative	Declarative	Event-driven
Learning curve	Low	High	Medium	Medium
State management	Idempotent tasks	Full state engine	State file (.tfstate)	Full state engine
Best for	Config mgmt, orchestration	Config mgmt at scale	Infrastructure provisioning	Event-driven automation
AWS integration	aws_ec2 module / dynamic inv.	CloudFormation	Native terraform-aws	cloud modules

★ Interview Tip
'Why Ansible over Terraform?' — Ansible is for configuration management (install packages, manage services, deploy apps). Terraform is for infrastructure provisioning (create VPCs, EC2, RDS). They complement each other. In production: Terraform provisions infra, Ansible configures it.

1.4 How Ansible Works — Execution Flow

Understanding this flow is critical for troubleshooting and interview questions:

1. Parse inventory	Reads hosts from static file or dynamic inventory script/plugin.
2. Read playbook	Parses YAML, validates syntax, resolves variables.
3. Gather facts	Connects to each host and collects system facts (unless gather_facts: false).
4. Generate modules	Converts each task into a Python module call.
5. Copy & execute	Copies module to /tmp on managed node, executes via Python, deletes it.
6. Return results	Module returns JSON result. Ansible processes changed/failed/ok.
7. Handlers	After all tasks, runs any notified handlers (e.g., restart nginx).

MODULE 2 — Inventory, Variables & Facts ■■■■■

2.1 Inventory — Static & Dynamic

Inventory is the source of truth for **what hosts** Ansible manages. Hosts can be grouped, grouped within groups, and assigned variables.

```
# inventory.ini / inventory.yml
# ■■ Static Inventory (INI format) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
[webservers]
web1.prod.example.com
web2.prod.example.com ansible_user=ec2-user ansible_port=22

[databases]
db1.prod.example.com ansible_host=10.0.1.50
db2.prod.example.com

[production:children]   # Group of groups
webservers
databases

[production:vars]       # Group variables
env=production
ansible_python_interpreter=/usr/bin/python3

# ■■ YAML format (preferred for complex inventories) ■■■■■■■■■■■■
all:
  children:
    webservers:
      hosts:
        web1.example.com:
          ansible_user: ec2-user
        web2.example.com:
      vars:
        http_port: 80
    databases:
      hosts:
        db1.example.com:
```

Dynamic Inventory (AWS EC2 Example)

Dynamic inventory queries cloud APIs at runtime to discover hosts. No manual host management — instances auto-appear/disappear.

```
# Dynamic inventory — AWS EC2
# Install AWS dynamic inventory plugin
pip install boto3 botocore

# aws_ec2.yml — dynamic inventory config
plugin: amazon.aws.aws_ec2
regions:
  - us-east-1
  - us-west-2
filters:
  instance-state-name: running
  tag:Environment: production
keyed_groups:
  - key: tags.Role
```

```
    prefix: role
  - key: placement.region
    prefix: aws_region
hostnames:
  - private-ip-address

# Use it:
ansible-playbook -i aws_ec2.yml site.yml
ansible-inventory -i aws_ec2.yml --list   # Preview hosts
```

2.2 Variable Precedence (Critical — Always Asked)

★★★★★ Most Important Concept
Variable precedence is the #1 most-asked Ansible interview topic. Know the order cold.
Higher number = higher priority = wins when conflict occurs.

Priority	Location	Example
22 (Highest)	**Extra vars (-e flag)**	**ansible-playbook site.yml -e 'env=prod'**
18	set_fact / registered vars	- set_fact: myvar: value
17	include_vars	- include_vars: secrets.yml
16	task vars (in task block)	vars: key: value inside a task
15	block vars	vars: under a block:
14	role vars (vars/main.yml)	Highest-priority role vars
13	play vars_files	vars_files: [common.yml]
12	play vars	vars: section in play
9	host_vars (file)	host_vars/web1.example.com.yml
8	group_vars (file)	group_vars/webservers.yml
5	Role defaults (defaults/main.yml)	Lowest-priority — easily overridden
1 (Lowest)	Inventory file vars	In [group:vars] sections

Rule of Thumb
Use role defaults/ for sensible defaults that can always be overridden.
Use group_vars/ and host_vars/ for environment-specific config.
Use -e flags or set_fact for one-off overrides or dynamic values.
NEVER store secrets in plain-text vars — use Ansible Vault.

2.3 Ansible Facts & Magic Variables

Facts are variables automatically discovered about managed nodes. Ansible collects them via the **setup** module at the start of every play (unless disabled).

```
# Facts & Magic Variables
# View all facts for a host
ansible web1 -m setup
ansible web1 -m setup -a 'filter=ansible_eth*'   # Filter facts
```

```
# Common fact variables (used in playbooks)
ansible_hostname         # Server hostname
ansible_os_family        # RedHat, Debian, Windows...
ansible_distribution     # CentOS, Ubuntu, Amazon...
ansible_distribution_version  # '22.04', '8.5', etc.
ansible_default_ipv4.address  # Primary IP
ansible_memtotal_mb      # Total RAM in MB
ansible_processor_vcpus  # vCPU count
ansible_architecture     # x86_64, aarch64...
ansible_env              # Environment variables dict

# Use in playbook
- name: Install package based on OS family
  package:
    name: nginx
    state: present
  when: ansible_os_family == 'Debian'

# Custom facts — place in /etc/ansible/facts.d/*.fact on managed node
# ansible_local.myapp.version → reads from /etc/ansible/facts.d/myapp.fact

# Magic Variables (not facts — built-in special vars)
inventory_hostname       # Current host's inventory name
inventory_hostname_short # Without domain
group_names              # List of groups host belongs to
groups                   # Dict of all groups and their hosts
hostvars                 # Access vars from other hosts
ansible_play_hosts       # Active hosts in current play
ansible_limit            # Value of --limit flag
```

★★★★★ How do you disable fact gathering and why would you?

Set **gather_facts: false** (or no) at the play level. This skips the setup module call.

Reasons: (1) Speed — fact gathering adds 1–3 seconds per host; on 200 hosts that's significant. (2) Hosts that don't need facts — simple ping checks, AWS API calls via localhost, etc. (3) Hosts that don't support fact gathering (some network devices).

Best practice: disable facts on plays that don't use ansible_* variables. Use gather_subset: ['min'] for a lightweight fact collection when you only need hostname/IP.

MODULE 3 — Playbooks & Tasks — Deep Dive ■■■■■

3.1 Playbook Structure

```
# Annotated production playbook
---
# site.yml — Production-quality playbook
- name: Configure web servers          # Play name
  hosts: webservers                    # Target group/host
  become: true                         # Privilege escalation (sudo)
  become_user: root
  gather_facts: true                   # Collect system facts
  serial: '20%'                        # Rolling update — 20% of hosts at a time
  max_fail_percentage: 10              # Abort if >10% hosts fail

  vars:                                # Play-level variables
    app_name: myapp
    app_version: '2.1.0'
    deploy_dir: /opt/{{ app_name }}

  vars_files:                          # Load external var files
    - vars/common.yml
    - vars/{{ ansible_os_family }}.yml

  pre_tasks:                           # Run BEFORE roles
    - name: Check disk space
      assert:
        that: ansible_mounts[0].size_available > 5368709120
        msg: 'Insufficient disk space'

  roles:                               # Apply roles
    - common
    - { role: nginx, tags: ['web'] }

  tasks:                               # Task list
    - name: Deploy application
      include_role:
        name: myapp

  post_tasks:                          # Run AFTER roles
    - name: Run smoke test
      uri:
        url: http://localhost/health
        status_code: 200

  handlers:                            # Triggered by notify
    - name: restart nginx
      service:
        name: nginx
        state: restarted
```

3.2 Task Control — Conditionals, Loops, Error Handling

```
# Task control patterns
# ■■ Conditionals (when) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
- name: Install Apache on RedHat only
```

```
  yum: name=httpd state=present
  when:
    - ansible_os_family == 'RedHat'
    - ansible_distribution_major_version | int >= 7

- name: Run only if variable is defined
  debug: msg='{{ myvar }}'
  when: myvar is defined and myvar != ''

# ■■ Loops ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
- name: Install multiple packages
  package:
    name: '{{ item }}'
    state: present
  loop:
    - nginx
    - curl
    - git

- name: Create users with loop and dict
  user:
    name: '{{ item.name }}'
    groups: '{{ item.groups }}'
    state: present
  loop:
    - { name: alice, groups: 'wheel,docker' }
    - { name: bob, groups: 'docker' }

- name: Loop with index (loop_control)
  debug: msg='Item {{ idx }}: {{ item }}'
  loop: ['a', 'b', 'c']
  loop_control:
    index_var: idx
    label: '{{ item }}'  # Clean output — shows only this in logs

# ■■ Error Handling ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
- name: Run a command that might fail
  command: /usr/bin/myscript.sh
  register: result
  ignore_errors: true              # Continue even on failure
  failed_when: result.rc not in [0, 2]  # Custom failure condition
  changed_when: result.stdout != ''      # Custom changed condition

- name: Fail with message
  fail:
    msg: 'Script failed with: {{ result.stderr }}'
  when: result.rc != 0

# ■■ Blocks for grouped error handling ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
- name: Block with rescue and always
  block:
    - name: Try to deploy
      command: /deploy.sh
    - name: Run migrations
      command: /migrate.sh
  rescue:
    - name: Roll back on failure
      command: /rollback.sh
    - name: Send alert
```

```
    uri:
      url: https://hooks.slack.com/services/xxx
      method: POST
      body_format: json
      body: {'text': 'Deployment failed on {{ inventory_hostname }}'}
  always:
    - name: Cleanup temp files
      file: path=/tmp/deploy state=absent
```

3.3 Register & Debug

```
# Register and Debug
# Register captures task output for later use
- name: Check if config file exists
  stat:
    path: /etc/myapp/config.yml
  register: config_stat

- name: Create config if missing
  template:
    src: config.yml.j2
    dest: /etc/myapp/config.yml
  when: not config_stat.stat.exists

# Register + loop = list of results
- name: Check services
  service_facts:
  register: services_state

- name: Print nginx status
  debug:
    msg: 'nginx is {{ services_state.ansible_facts.services["nginx.service"].state }}'

# Debug verbosity levels
- debug: msg='Always shown'
- debug: msg='Shown at -vv' verbosity=2
- debug: var=myvar   # Pretty-prints variable
```

3.4 Handlers

Handlers are tasks that run only when notified by other tasks AND only once per play, at the end of the play. They prevent restarting services multiple times.

```
# Handlers pattern
tasks:
  - name: Update nginx config
    template:
      src: nginx.conf.j2
      dest: /etc/nginx/nginx.conf
    notify: restart nginx           # Queues handler

  - name: Update SSL certificate
    copy:
      src: cert.pem
      dest: /etc/ssl/cert.pem
    notify:                         # Multiple notifications
      - restart nginx
      - reload systemd
```

```
handlers:
  - name: restart nginx
    service:
      name: nginx
      state: restarted

  - name: reload systemd
    systemd:
      daemon_reload: true

# Force handler flush mid-play
- meta: flush_handlers
```

Handler Gotchas
Handlers run ONCE at the end, even if notified 10 times — this is the feature, not a bug.
If a play fails midway, handlers WON'T run (use --force-handlers to override).
Handler names must match EXACTLY — case-sensitive.
listen: allows multiple tasks to trigger one handler with different notify names.

3.5 Tags, Delegation & Async

```
# Tags, delegation, async
# ■■ Tags — run subsets of tasks ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
- name: Install packages
  package: name=nginx state=present
  tags: [packages, install]

- name: Deploy config
  template: src=nginx.conf.j2 dest=/etc/nginx/nginx.conf
  tags: [config, deploy]

# Run only tagged tasks:
ansible-playbook site.yml --tags config
ansible-playbook site.yml --skip-tags packages
ansible-playbook site.yml --tags all  # Always run (special tag)

# ■■ Delegation — run task on different host ■■■■■■■■■■■■■■■■■■■■
- name: Register instance in load balancer
  shell: aws elbv2 register-targets ...
  delegate_to: localhost             # Run on control node

- name: Take DB snapshot before upgrade
  shell: pg_dump -h {{ inventory_hostname }} mydb > /backup/db.sql
  delegate_to: backup-server         # Run on a different managed node

# ■■ Async tasks — don't wait for long-running tasks ■■■■■■■■■■
- name: Run long job async
  command: /opt/scripts/long_process.sh
  async: 3600          # Max run time in seconds
  poll: 0              # Don't wait — fire and forget
  register: async_job

- name: Check job status
  async_status:
    jid: '{{ async_job.ansible_job_id }}'
  register: job_result
  until: job_result.finished
  retries: 60
  delay: 10
```

MODULE 4 — Roles, Collections & Galaxy

4.1 Role Structure

Roles are the primary way to package and reuse Ansible content. A role is a directory structure with a defined layout — Ansible knows where to look for tasks, variables, templates, files, and handlers automatically.

```
# Role directory structure
roles/
  nginx/                    # Role name
    tasks/
      main.yml              # Entry point — all tasks go here (or include others)
      install.yml           # Split tasks into files, include from main.yml
      configure.yml
    handlers/
      main.yml              # Handlers for this role
    templates/              # Jinja2 templates (.j2 files)
      nginx.conf.j2
      vhost.conf.j2
    files/                  # Static files (no template processing)
      nginx_logo.png
    vars/
      main.yml              # High-priority vars (hard to override)
    defaults/
      main.yml              # Low-priority defaults (easily overridden — prefer this)
    meta/
      main.yml              # Role metadata — dependencies, Galaxy info
    library/                # Custom modules for this role
    module_utils/           # Custom module utilities
    lookup_plugins/         # Custom lookup plugins
    tests/
      inventory             # Test inventory
      test.yml              # Test playbook
    README.md
```

```
# Role defaults, meta, and usage
# roles/nginx/defaults/main.yml — easily overridable defaults
nginx_port: 80
nginx_worker_processes: auto
nginx_max_body_size: 10m
nginx_ssl_enabled: false
nginx_conf_dir: /etc/nginx/conf.d

# roles/nginx/meta/main.yml — role dependencies
galaxy_info:
  author: yourname
  description: Install and configure Nginx
  license: MIT
  min_ansible_version: '2.12'
  platforms:
    - name: Ubuntu
      versions: [focal, jammy]
    - name: EL
      versions: [8, 9]
dependencies:
  - role: common         # This role runs first
  - role: geerlingguy.firewall
    vars:
```

```
    firewall_allowed_tcp_ports: [80, 443]

# Using a role in a playbook
- hosts: webservers
  roles:
    - common
    - { role: nginx, nginx_port: 8080, tags: web }
    - role: nginx
      vars:                    # Pass role vars
        nginx_port: 8443
        nginx_ssl_enabled: true
      when: enable_ssl | bool
```

4.2 Collections

Collections are the modern packaging format for Ansible content — roles, modules, plugins, and playbooks bundled together with versioning. Ansible Galaxy distributes both roles and collections.

 # Collections and Galaxy

```
# requirements.yml — declare dependencies (like package.json)
---
collections:
  - name: amazon.aws          # AWS modules
    version: '>=6.0.0'
  - name: community.general
    version: '>=7.0.0'
  - name: ansible.posix
  - name: community.crypto    # TLS/SSL modules

roles:
  - name: geerlingguy.nginx
    version: 3.1.0
  - src: https://github.com/myorg/myrole
    scm: git
    version: main

# Install all dependencies
ansible-galaxy install -r requirements.yml
ansible-galaxy collection install -r requirements.yml

# Install to specific path (for project isolation)
ansible-galaxy collection install amazon.aws -p ./collections

# Use collection module in playbook
- name: Create EC2 instance
  amazon.aws.ec2_instance:
    name: my-server
    instance_type: t3.medium
    image_id: ami-0abcdef1234567890
    vpc_subnet_id: subnet-12345
    security_groups: [sg-12345]
    region: us-east-1

# ansible.cfg — set collection path
[defaults]
collections_paths = ./collections:~/.ansible/collections
```

MODULE 5 — Modules — Essential & Advanced ■■■■■

5.1 File & System Modules

```
# File and system modules
# ■■ file — manage files, dirs, symlinks, permissions ■■■■■■■■■■
- name: Create directory with permissions
  file:
    path: /opt/myapp/logs
    state: directory
    owner: appuser
    group: appgroup
    mode: '0755'
    recurse: true

- name: Create symlink
  file:
    src: /opt/myapp/current
    dest: /usr/local/bin/myapp
    state: link

# ■■ copy — copy files to remote ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
- name: Copy with backup
  copy:
    src: files/myconfig.conf
    dest: /etc/myapp/config.conf
    owner: root
    group: root
    mode: '0640'
    backup: true      # Creates backup with timestamp

# ■■ template — copy with Jinja2 rendering ■■■■■■■■■■■■■■■■■■■
- name: Render and deploy config template
  template:
    src: templates/myapp.conf.j2
    dest: /etc/myapp/myapp.conf
    owner: appuser
    mode: '0600'
    validate: '/usr/sbin/myapp --check %s'  # Validate before deploy
  notify: restart myapp

# ■■ lineinfile — ensure a line exists in file ■■■■■■■■■■■■■■■■
- name: Ensure sysctl setting
  lineinfile:
    path: /etc/sysctl.conf
    regexp: '^net.ipv4.ip_forward'
    line: 'net.ipv4.ip_forward = 1'
    state: present
    backup: true

# ■■ blockinfile — manage a block of lines ■■■■■■■■■■■■■■■■■■■
- name: Add SSH keys block
  blockinfile:
    path: /etc/ssh/sshd_config
    block: |
      Match Group sre-team
        PasswordAuthentication no
```

```
        AllowTcpForwarding yes
    marker: '# {mark} ANSIBLE MANAGED BLOCK sre-team'

# ■■ fetch — download files from remote ■■■■■■■■■■■■■■■■■■■■■■■
- name: Fetch log file
  fetch:
    src: /var/log/myapp/error.log
    dest: ./fetched_logs/
    flat: false  # Preserves host directory structure

# ■■ find — find files matching criteria ■■■■■■■■■■■■■■■■■■■■■
- name: Find old log files
  find:
    paths: /var/log/myapp
    patterns: '*.log'
    age: 30d       # Older than 30 days
    recurse: true
  register: old_logs

- name: Delete old logs
  file: path={{ item.path }} state=absent
  loop: '{{ old_logs.files }}'
```

5.2 Package & Service Modules

```
# Package and service modules
# ■■ package — OS-agnostic package manager ■■■■■■■■■■■■■■■■■■■
- name: Install packages (works on any OS)
  package:
    name: ['nginx', 'curl', 'htop']
    state: present

# ■■ yum/dnf — RedHat family ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
- name: Install and enable EPEL
  yum:
    name: epel-release
    state: present

- name: Update all packages
  dnf:
    name: '*'
    state: latest
    exclude: kernel*   # Don't update kernel

# ■■ apt — Debian family ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
- name: Add nginx apt key and repo
  block:
    - apt_key:
        url: https://nginx.org/keys/nginx_signing.key
        state: present
    - apt_repository:
        repo: 'deb http://nginx.org/packages/ubuntu {{ ansible_distribution_release }} nginx
'
        state: present
    - apt:
        name: nginx
        state: present
        update_cache: true
```

```
# ■■ service — manage services ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
- name: Enable and start nginx
  service:
    name: nginx
    state: started
    enabled: true

# ■■ systemd — advanced systemd management ■■■■■■■■■■■■■■■■■■
- name: Deploy and enable custom service
  block:
    - template:
        src: myapp.service.j2
        dest: /etc/systemd/system/myapp.service
    - systemd:
        name: myapp
        state: started
        enabled: true
        daemon_reload: true   # Reload systemd after unit change
```

5.3 Command Execution Modules

 # Command execution — command vs shell vs raw

```
# command vs shell vs script vs raw — know the difference!

# ■■ command — preferred for simple commands ■■■■■■■■■■■■■■■
# Does NOT use shell — no pipes, redirects, env expansion
- name: Check if process is running
  command: pgrep nginx
  register: nginx_pid
  changed_when: false    # command always shows 'changed' — override
  failed_when: nginx_pid.rc not in [0, 1]

# ■■ shell — use when you need shell features ■■■■■■■■■■■■■■
# Supports pipes, redirects, globs, env vars
- name: Get disk usage
  shell: df -h | grep /dev/xvda | awk '{print $5}'
  register: disk_usage
  changed_when: false

# ■■ script — copy and run local script on remote ■■■■■■■■■■
- name: Run deployment script
  script: scripts/deploy.sh {{ app_version }}
  args:
    creates: /opt/myapp/{{ app_version }}  # Idempotent — skip if path exists

# ■■ raw — no Python required on remote ■■■■■■■■■■■■■■■■■■■■
# Use only for bootstrapping (before Python is installed)
- name: Install Python on bare server
  raw: apt-get update && apt-get install -y python3
  changed_when: true

# ■■ args: creates/removes — make command idempotent ■■■■■■■
- name: Extract archive (only if dir doesn't exist)
  command: tar xzf /tmp/app-{{ version }}.tgz -C /opt
  args:
    creates: /opt/app-{{ version }}  # Skip if this path exists
```

5.4 Network & Cloud Modules

```
# Network, wait, and AWS modules
# ■■ uri — make HTTP requests ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
- name: Health check
  uri:
    url: http://localhost:8080/health
    method: GET
    status_code: [200, 204]
    timeout: 30
  register: health
  until: health.status == 200
  retries: 10
  delay: 5

- name: POST to API
  uri:
    url: https://api.example.com/deployments
    method: POST
    headers:
      Authorization: 'Bearer {{ api_token }}'
      Content-Type: application/json
    body_format: json
    body:
      version: '{{ app_version }}'
      environment: production
    status_code: 201

# ■■ wait_for — wait for ports/files/conditions ■■■■■■■■■■■■■■■■
- name: Wait for app to start
  wait_for:
    host: localhost
    port: 8080
    state: started
    delay: 5
    timeout: 120

- name: Wait for file to appear
  wait_for:
    path: /tmp/deployment.done
    state: present
    timeout: 300

# ■■ AWS modules (amazon.aws collection) ■■■■■■■■■■■■■■■■■■■■■■■■
- name: Get EC2 instance info
  amazon.aws.ec2_instance_info:
    filters:
      'tag:Environment': production
      instance-state-name: running
  register: ec2_instances

- name: Create S3 bucket
  amazon.aws.s3_bucket:
    name: myapp-artifacts-{{ env }}
    region: us-east-1
    versioning: true
    encryption: 'aws:kms'
    state: present

- name: Upload to S3
```

```
amazon.aws.aws_s3:
  bucket: myapp-artifacts-{{ env }}
  object: /releases/{{ app_version }}.tar.gz
  src: /tmp/{{ app_version }}.tar.gz
  mode: put
```

MODULE 6 — Templating (Jinja2) & Filters ■■■■

6.1 Jinja2 in Ansible

Ansible uses Jinja2 as its templating engine — in template files (.j2), variable values, conditionals, and anywhere {{ }} or {% %} appear. Master these to write dynamic playbooks.

```
# nginx.conf.j2 — real-world template example
# ■■ Template file: nginx.conf.j2 ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
worker_processes {{ ansible_processor_vcpus }};

events {
    worker_connections {{ nginx_worker_connections | default(1024) }};
}

http {
    server_tokens off;
    client_max_body_size {{ nginx_max_body_size | default('10m') }};

    {% for vhost in nginx_vhosts %}
    server {
        listen {{ vhost.port | default(80) }};
        server_name {{ vhost.name }};
        root {{ vhost.root | default('/var/www/html') }};

        {% if vhost.ssl | default(false) %}
        listen 443 ssl;
        ssl_certificate /etc/ssl/{{ vhost.name }}.crt;
        ssl_certificate_key /etc/ssl/{{ vhost.name }}.key;
        {% endif %}

        {% if vhost.auth | default(false) %}
        auth_basic 'Restricted';
        auth_basic_user_file /etc/nginx/.htpasswd;
        {% endif %}
    }
    {% endfor %}

    # Upstream from inventory
    upstream backend {
        {% for host in groups['appservers'] %}
        server {{ hostvars[host]['ansible_default_ipv4']['address'] }}:{{ app_port }};
        {% endfor %}
    }
}
```

6.2 Essential Jinja2 Filters

```
# Essential Jinja2 filters
# ■■ String filters ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
{{ myvar | upper }}              # UPPERCASE
{{ myvar | lower }}              # lowercase
{{ myvar | title }}              # Title Case
{{ myvar | trim }}               # Remove leading/trailing whitespace
{{ myvar | replace('old','new')}} # Replace string
{{ myvar | truncate(50) }}       # Truncate to 50 chars
{{ myvar | regex_replace('^#.*$', '', multiline=True) }}
```

```
# ■■ Default values ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
{{ myvar | default('fallback') }}           # If undefined
{{ myvar | default('fallback', true) }}     # If undefined OR empty
{{ myvar | default(omit) }}                 # Omit the param if undefined

# ■■ Type conversion ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
{{ '3' | int }}                  # String to integer
{{ '3.14' | float }}             # String to float
{{ myvar | string }}             # Anything to string
{{ myvar | bool }}               # To boolean
{{ myvar | list }}               # To list
{{ myvar | to_json }}            # To JSON string
{{ myvar | to_yaml }}            # To YAML string
{{ myvar | from_json }}          # Parse JSON string

# ■■ List/dict filters ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
{{ mylist | length }}            # Count items
{{ mylist | first }}             # First item
{{ mylist | last }}              # Last item
{{ mylist | unique }}            # Remove duplicates
{{ mylist | sort }}              # Sort list
{{ mylist | join(', ') }}        # Join to string
{{ mylist | select('match', '^prod.*') | list }}  # Filter by pattern
{{ mylist | map(attribute='name') | list }}       # Extract attribute
{{ mydict | dict2items }}        # Dict to list of {key,value} dicts
{{ items | items2dict }}         # Back to dict
{{ mydict | combine({'new_key': 'val'}) }}        # Merge dicts

# ■■ Path & file filters ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
{{ '/etc/nginx/nginx.conf' | basename }}   # nginx.conf
{{ '/etc/nginx/nginx.conf' | dirname }}    # /etc/nginx
{{ myfile | expanduser }}        # Expand ~

# ■■ Crypto & hash ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
{{ 'mysecret' | password_hash('sha512') }}  # Hash password (for user module)
{{ myvar | hash('sha1') }}
{{ myvar | b64encode }}          # Base64 encode
{{ myvar | b64decode }}          # Base64 decode

# ■■ Conditional filters ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
{{ myvar | ternary('yes_val', 'no_val') }}  # Inline if/else
{{ mylist | selectattr('active', 'equalto', true) | list }}
{{ mylist | rejectattr('name', 'match', 'temp.*') | list }}
```

MODULE 7 — Ansible Vault & Secrets Management ■■■■■

7.1 Ansible Vault Commands

Ansible Vault encrypts sensitive data at rest using AES-256 encryption. Encrypted files are decrypted automatically at runtime when the vault password is provided.

```
# Ansible Vault commands
# ■■ Create and edit encrypted files ■■■■■■■■■■■■■■■■■■■■■■■■■
ansible-vault create secrets.yml          # Create new encrypted file
ansible-vault edit secrets.yml            # Edit encrypted file
ansible-vault view secrets.yml            # View without editing
ansible-vault encrypt existing.yml        # Encrypt existing file
ansible-vault decrypt secrets.yml         # Decrypt (careful in prod!)
ansible-vault rekey secrets.yml           # Change password

# ■■ Encrypt a single string (inline vault) ■■■■■■■■■■■■■■■■■■■
ansible-vault encrypt_string 'mysecretpassword' --name 'db_password'
# Output:
db_password: !vault |
  $ANSIBLE_VAULT;1.1;AES256
  66386439653236336462626566653265353...

# Use inline vault in vars file:
db_password: !vault |
  $ANSIBLE_VAULT;1.1;AES256
  66386439653236336462626566653265353...

# ■■ Multiple vault IDs (separate passwords per environment) ■
ansible-vault create --vault-id prod@prompt prod_secrets.yml
ansible-vault create --vault-id dev@prompt dev_secrets.yml

# Run playbook with multiple vault passwords
ansible-playbook site.yml --vault-id prod@~/.vault_pass_prod \
                          --vault-id dev@~/.vault_pass_dev

# ■■ Password file (recommended for CI/CD) ■■■■■■■■■■■■■■■■■■■■
echo 'myvaultpassword' > ~/.vault_pass
chmod 600 ~/.vault_pass

# ansible.cfg
[defaults]
vault_password_file = ~/.vault_pass

# ■■ Run with vault password ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ansible-playbook site.yml --vault-password-file ~/.vault_pass
ansible-playbook site.yml --ask-vault-pass   # Prompt interactively
```

7.2 Vault Best Practices (Production)

```
# Vault best practices and integrations
# group_vars/production/
#   vars.yml           — plain text variables
#   vault.yml          — encrypted secrets (name vars with vault_ prefix)

# group_vars/production/vault.yml (encrypted)
vault_db_password: supersecret123
```

```
vault_api_key: abc123xyz
vault_ssl_cert: |
  -----BEGIN CERTIFICATE-----
  ...

# group_vars/production/vars.yml (plain text — references vault vars)
db_password: '{{ vault_db_password }}'  # Reference, not the secret itself
api_key: '{{ vault_api_key }}'

# Why this pattern?
# - You can grep for variable NAMES without decrypting
# - git diff shows which secret changed (encrypted hash changes)
# - Clear separation of config vs secrets

# ■■ AWS Secrets Manager integration (alternative to Vault) ■■
# Use community.aws.aws_secret lookup plugin:
- name: Get secret from AWS Secrets Manager
  vars:
    db_pass: "{{ lookup('amazon.aws.aws_secret', 'prod/myapp/db_password') }}"
  debug:
    msg: 'Connected to DB'   # Use db_pass in tasks

# ■■ HashiCorp Vault integration ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
- name: Get secret from HashiCorp Vault
  vars:
    secret: "{{ lookup('community.hashi_vault.hashi_vault',
               'secret=secret/data/myapp:db_password
                url=https://vault.example.com:8200
                auth_method=aws_iam') }}"
```

Critical Security Rules

NEVER commit unencrypted vault files to git. Add *.yml to .gitignore if needed, or only add vault.yml.
NEVER use the same vault password across environments — use vault IDs.
In CI/CD: store vault password in CI secrets (GitHub Actions secrets, Jenkins credentials, AWS SSM).
Rotate vault passwords periodically using ansible-vault rekey.
Use --vault-id with named IDs in multi-environment pipelines to avoid decrypting wrong secrets.

MODULE 8 — Performance, Optimization & Scaling ■■■■

8.1 Parallelism & Forks

Ansible runs tasks on multiple hosts simultaneously. The **forks** setting controls how many hosts are processed in parallel. Default is 5 — far too low for large fleets.

Performance tuning — forks, pipelining, serial

```
# ansible.cfg — performance tuning
[defaults]
forks = 50                    # Run on 50 hosts in parallel (default: 5)
gathering = smart             # Cache facts — re-gather only if missing
fact_caching = jsonfile
fact_caching_connection = /tmp/ansible_fact_cache
fact_caching_timeout = 7200  # 2 hours cache

# Pipelining — reduces SSH connections per task
[ssh_connection]
pipelining = True             # BIG performance boost — requires requiretty disabled
ssh_args = -o ControlMaster=auto -o ControlPersist=60s  # SSH multiplexing
control_path_dir = /tmp/.ansible/cp

# ■■ Serial — rolling updates ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# In playbook — control blast radius
- hosts: webservers
  serial: 1           # One host at a time (safest)
  serial: 5           # Five hosts at a time
  serial: '20%'       # 20% of inventory at a time
  serial:             # Progressive — 1, then 5, then all remaining
    - 1
    - 5
    - 100%

# ■■ max_fail_percentage ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
- hosts: webservers
  serial: '25%'
  max_fail_percentage: 5  # Abort if more than 5% of hosts fail

# ■■ Strategy plugins ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
- hosts: all
  strategy: free       # Don't wait for slowest host — each host races ahead
  # Default: linear (all hosts complete each task before next starts)
  # host_pinned: like free but stays on one host at a time
```

8.2 Fact Caching & Optimization Techniques

Optimization techniques

```
# ■■ Disable facts when not needed ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
- hosts: all
  gather_facts: false
  tasks:
    - ping:   # Just test connectivity — facts not needed

# ■■ gather_subset — collect only needed facts ■■■■■■■■■■■■■■
- hosts: all
  gather_subset:
    - '!all'        # Don't gather anything
```

```
    - '!any'
    - network       # Only gather network facts
    - min           # Minimal — hostname, OS, fqdn only

# ■■ Mitogen strategy (massive speedup) ■■■■■■■■■■■■■■■■■■■■■■
# Install: pip install mitogen
# ansible.cfg:
# [defaults]
# strategy_plugins = /path/to/mitogen/ansible_mitogen/plugins/strategy
# strategy = mitogen_linear
# Typical speedup: 3-10x faster than default strategy

# ■■ Use --limit to run on subsets ■■■■■■■■■■■■■■■■■■■■■■■■■■
ansible-playbook site.yml --limit webservers     # Only webservers
ansible-playbook site.yml --limit 'web1,web2'   # Specific hosts
ansible-playbook site.yml --limit '!db*'        # All except db* hosts

# ■■ Check mode and diff ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ansible-playbook site.yml --check          # Dry run — no changes
ansible-playbook site.yml --check --diff   # Show what WOULD change
ansible-playbook site.yml --diff           # Show changes as they happen

# ■■ Step mode (interactive) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ansible-playbook site.yml --step           # Confirm each task
ansible-playbook site.yml --start-at-task 'Deploy app'  # Skip to task
```

MODULE 9 — Ansible in CI/CD & AWS

9.1 GitHub Actions with Ansible

```
# GitHub Actions CI/CD pipeline
```

```
# .github/workflows/deploy.yml
name: Deploy with Ansible
on:
  push:
    branches: [main]
  workflow_dispatch:
    inputs:
      environment:
        description: 'Environment (staging/production)'
        required: true
        default: staging

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: ${{ github.event.inputs.environment || 'staging' }}
    steps:
      - uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install Ansible
        run: |
          pip install ansible boto3 botocore
          ansible-galaxy install -r requirements.yml
          ansible-galaxy collection install -r requirements.yml

      - name: Configure SSH
        run: |
          mkdir -p ~/.ssh
          echo '${{ secrets.SSH_PRIVATE_KEY }}' > ~/.ssh/id_rsa
          chmod 600 ~/.ssh/id_rsa
          ssh-keyscan -H ${{ secrets.BASTION_HOST }} >> ~/.ssh/known_hosts

      - name: Write vault password
        run: echo '${{ secrets.ANSIBLE_VAULT_PASSWORD }}' > ~/.vault_pass

      - name: Run Ansible playbook
        env:
          AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
          AWS_SECRET_ACCESS_KEY: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          AWS_DEFAULT_REGION: us-east-1
        run: |
          ansible-playbook site.yml \
            -i inventories/${{ github.event.inputs.environment || 'staging' }} \
            --vault-password-file ~/.vault_pass \
            -e app_version=${{ github.sha }} \
            -e env=${{ github.event.inputs.environment || 'staging' }} \
            --diff
```

```
    - name: Cleanup
      if: always()
      run: |
        rm -f ~/.ssh/id_rsa ~/.vault_pass
```

9.2 AWS Integration Patterns

```
# AWS integration patterns
# ■■ Pattern 1: EC2 provisioning + configuration ■■■■■■■■■■■■■■■
- name: Provision and configure EC2 instances
  hosts: localhost
  gather_facts: false
  tasks:
    - name: Launch EC2 instance
      amazon.aws.ec2_instance:
        name: '{{ app_name }}-{{ env }}'
        instance_type: '{{ instance_type | default("t3.medium") }}'
        image_id: '{{ ami_id }}'
        vpc_subnet_id: '{{ subnet_id }}'
        security_groups: ['{{ sg_id }}']
        iam_instance_profile: '{{ iam_profile }}'
        key_name: '{{ key_pair }}'
        tags:
          Name: '{{ app_name }}-{{ env }}'
          Environment: '{{ env }}'
          ManagedBy: Ansible
        wait: true
        wait_timeout: 300
      register: ec2_result

    - name: Add to dynamic in-memory inventory
      add_host:
        hostname: '{{ ec2_result.instances[0].private_ip_address }}'
        groups: newly_provisioned
        ansible_user: ec2-user
        ansible_ssh_private_key_file: '{{ key_file }}'

- name: Configure new instances
  hosts: newly_provisioned
  become: true
  roles:
    - common
    - '{{ app_role }}'

# ■■ Pattern 2: SSM Session Manager (no SSH needed) ■■■■■■■■■■
# ansible.cfg
[defaults]
remote_user = ssm-user

[ssh_connection]
ssh_executable = /usr/bin/ssh
ssh_args = -o ProxyCommand='aws ssm start-session --target %h \
  --document-name AWS-StartSSHSession --parameters portNumber=%p'

# inventory/hosts
i-0123456789abcdef0 ansible_host=i-0123456789abcdef0

# ■■ Pattern 3: ECS task deployment via Ansible ■■■■■■■■■■■■■■■■
```

```
- name: Deploy new ECS task definition
  community.aws.ecs_taskdefinition:
    family: myapp
    containers:
      - name: web
        image: '{{ ecr_repo }}/myapp:{{ app_version }}'
        portMappings:
          - containerPort: 8080
        environment:
          - name: ENV
            value: '{{ env }}'
        secrets:
          - name: DB_PASSWORD
            valueFrom: 'arn:aws:secretsmanager:{{ region }}:{{ account }}:secret:db_pass'
        logConfiguration:
          logDriver: awslogs
          options:
            awslogs-group: '/ecs/myapp'
            awslogs-region: '{{ region }}'
    requires_compatibilities: [FARGATE]
    network_mode: awsvpc
    cpu: '256'
    memory: '512'
  register: task_def

- name: Update ECS service
  community.aws.ecs_service:
    name: myapp-service
    cluster: myapp-cluster
    task_definition: '{{ task_def.taskdefinition.taskDefinitionArn }}'
    desired_count: 3
    deployment_configuration:
      minimum_healthy_percent: 100
      maximum_percent: 200
  register: svc_result
```

MODULE 10 — Production Troubleshooting ■■■■■

10.1 Debugging Approach

```
# Debugging commands
# ■■ Verbosity levels ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ansible-playbook site.yml -v    # Basic — show task results
ansible-playbook site.yml -vv   # Show files used, task args
ansible-playbook site.yml -vvv  # SSH connection details
ansible-playbook site.yml -vvvv # Connection plugin info + SSH debug

# ■■ Syntax check ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ansible-playbook site.yml --syntax-check
ansible-lint site.yml           # Best practices lint

# ■■ List what will run ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ansible-playbook site.yml --list-tasks    # List all tasks
ansible-playbook site.yml --list-hosts    # List target hosts
ansible-playbook site.yml --list-tags     # List all tags

# ■■ Test connectivity ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ansible all -m ping
ansible webservers -m ping -v
ansible all -m setup -a 'filter=ansible_hostname'  # Spot check

# ■■ Run ad-hoc commands for diagnostics ■■■■■■■■■■■■■■■■■■■■■■■■
ansible web1 -m shell -a 'df -h'
ansible web1 -m shell -a 'systemctl status nginx'
ansible all -m shell -a 'free -m' --one-line

# ■■ Environment variable debug ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ANSIBLE_DEBUG=1 ansible-playbook site.yml      # Full Python debug
ANSIBLE_KEEP_REMOTE_FILES=1 ansible-playbook site.yml  # Don't delete remote modules
```

10.2 Common Production Scenarios

SSH	**■ SSH Connection Refused / Timeout** • ansible all -m ping -vvv → check SSH errors • Check: correct ansible_user? Correct SSH key? Port 22 open in SG? • Test manually: ssh -i key.pem ec2-user@host • Check StrictHostKeyChecking: add host_key_checking = False in ansible.cfg (dev only!) • For AWS: ensure bastion/SSM proxy configured, instance has correct IAM role
Idempotent	**■ Playbook says 'changed' every run — not idempotent** • Use command/shell? → add creates: or removes: args, or changed_when: false • Template always regenerating? → check for dynamic content (timestamps, random values) • File permissions mismatch? → check mode/owner/group in file tasks • Use --check --diff to see what's changing without applying • Prefer idempotent modules (package, service, template) over shell commands

Variables	■ **Variable undefined error** • Check variable precedence — is it defined at the right level? • Use | default('value') filter to provide fallbacks • Check group_vars/ dir matches group name exactly (case-sensitive!) • ansible-playbook --extra-vars 'myvar=test' to test quickly • Use ansible -m debug -a 'var=hostvars[inventory_hostname]' to dump all vars
Partial Failure	■ **Playbook fails on some hosts but succeeds on others** • Check ansible_os_family / ansible_distribution — different OS handling needed • Enable serial: 1 to isolate failures • Use --limit hostname to reproduce on failing host • Check if host is in multiple groups with conflicting vars • Review ignore_errors and failed_when conditions — may be masking failures
Handlers	■ **Handlers not running after task** • Handler name must match EXACTLY — check for typos and case • Was the task actually 'changed'? Handlers only trigger on changed, not ok • Did play fail before handler section ran? Use --force-handlers • Handler may be in wrong scope (role vs play handlers) • Add - meta: flush_handlers to force immediate handler execution
Inventory	■ **'No hosts matched' error** • Check inventory file path and format • ansible-inventory -i inventory/ --list to preview hosts • Check if --limit is filtering everything out • Dynamic inventory: check AWS credentials, region, filters • Verify group names in playbook match inventory group names exactly
Vault	■ **Vault decryption errors** • Wrong password? Try ansible-vault view file.yml to test • Multiple vault IDs? Ensure correct --vault-id is specified • File not actually encrypted? Check it starts with $ANSIBLE_VAULT • vault_password_file path wrong in ansible.cfg? • Whitespace/newline in password file? Use echo -n 'pass' > ~/.vault_pass
Performance	■ **Slow playbook execution** • Enable pipelining = True in ansible.cfg (biggest single win) • Increase forks = 50 for parallel execution • Enable fact caching — skip re-gathering on every run • Add gather_facts: false to plays that don't use facts • Use --tags to run only changed components • Consider Mitogen strategy plugin (3-10x speedup)

MODULE 11 — Top Interview Questions & Answers

★★★★★ Must-Know Questions (Asked in Nearly Every Interview)

★★★★★ What is Ansible and how does it differ from Puppet/Chef?

Ansible is an agentless automation tool using SSH/WinRM — no agents on managed nodes. Ansible uses push-based model; Puppet/Chef use pull-based (agents poll server). Ansible uses simple YAML (playbooks); Puppet/Chef use custom DSLs (Ruby). Ansible has a lower learning curve and no master server required. Puppet/Chef better for continuous enforcement at scale; Ansible better for orchestration and ad-hoc automation.

★★★★★ Explain Ansible playbook structure — all key sections.

Playbook contains one or more plays. Each play has: hosts (target), become (privilege escalation), gather_facts, vars/vars_files, pre_tasks, roles, tasks, post_tasks, and handlers. Tasks are the core unit — each maps to a module call. Handlers are tasks triggered by notify and run once at play end. Roles organize tasks into reusable units with their own vars, files, templates, handlers.

★★★★★ What is idempotency and why does it matter in Ansible?

Idempotency means running the same operation multiple times produces the same result — no unintended side effects on subsequent runs. Ansible built-in modules (package, file, service, template) are idempotent. Shell/command modules are NOT inherently idempotent — use creates:/removes: or changed_when: false. Idempotency enables safe re-runs for drift correction, retries after failures, and confidence in automation.

★★★★★ Explain Ansible variable precedence — which has highest priority?

22 levels total. Extra vars (-e flag) has HIGHEST priority. Then: set_fact, include_vars, task vars, block vars, role vars/main.yml, play vars_files, play vars. Then: host_vars files, group_vars files. LOWEST: role defaults/main.yml, inventory file vars. Key insight: role defaults are INTENTIONALLY lowest — they're meant to be overridden. Use -e for one-off overrides, group_vars for env-specific config, defaults for role defaults.

★★★★★ What is the difference between roles and collections?

A Role is a reusable, structured directory of tasks, vars, handlers, templates for ONE function. A Collection is a packaging format that can contain multiple roles, custom modules, plugins, and playbooks — distributed and versioned as a unit on Ansible Galaxy. Think of a role like a library and a collection like a package (npm package containing multiple libraries). Use collections to distribute Ansible content for complex platforms (amazon.aws, community.general).

★★★★★ How does Ansible handle secrets? What is Ansible Vault?

Ansible Vault encrypts sensitive data using AES-256. Can encrypt whole files or individual strings. ansible-vault create/edit/encrypt/decrypt. Use --vault-password-file in CI/CD. Best practice: separate vault.yml (encrypted) from vars.yml (plain), use vault_ prefix for encrypted vars. Use vault IDs for multiple environments. Alternatives: integrate with AWS Secrets Manager (aws_secret lookup), HashiCorp Vault (hashi_vault lookup).

★★★★★ What is dynamic inventory and when would you use it?

Static inventory lists hosts in a file — doesn't scale in cloud environments where hosts change. Dynamic inventory queries cloud APIs at runtime (AWS EC2, GCP, Azure, etc.) to discover hosts. Configured via inventory plugins (aws_ec2.yml) or scripts. Groups hosts by tags, instance type, region automatically. Use in any cloud environment where servers scale up/down — no manual inventory maintenance.

★★★★★ What is the difference between import and include in Ansible?

import_* (import_tasks, import_playbook, import_role): STATIC — processed at parse time. Tags applied to import are inherited by all tasks inside. Cannot use variables in the file name. include_* (include_tasks, include_role): DYNAMIC — processed at runtime. Can use variables in file names. Tags must be specified on each task. Use import for static, known structures. Use include for conditional, dynamic task loading. Key gotcha: with_items/loop only works with include_tasks, not import_tasks.

★★★★★ What are handlers and when do they run?

Handlers are tasks that run when notified by another task AND only when that task reports 'changed'. They run ONCE per play at the END, regardless of how many times notified. Common use: restart/reload services after config changes. Gotchas: run only if play completes (use --force-handlers to override), name must match exactly, use meta: flush_handlers to trigger mid-play.

★★★★★ How do you implement a rolling deployment with Ansible?

Use serial: to limit how many hosts are updated at once. serial: '20%' processes 20% of hosts per batch. serial: [1, 5, 100%] is progressive — 1 host, then 5, then all. Combine with max_fail_percentage: to abort if too many failures. Pattern: pre_tasks check drain from LB → roles deploy → post_tasks health check → handlers restart. This ensures the fleet is never fully down during deployment.

★★★★★ Explain the command, shell, and raw modules — when to use each?

command: safest — runs command directly without shell. No pipes, redirects, or shell expansion. Does not spawn a shell process. Always prefer command when no shell features needed. shell: spawns /bin/sh — supports pipes, redirects, globs, env vars. Use when shell features needed. raw: no Python required on managed node — sends raw SSH commands. Use only for bootstrapping (e.g., installing Python before Ansible can run normally). Both command and shell show 'changed' every run — use changed_when: false to override.

★★★★★ What is become and how does privilege escalation work?

become: true enables privilege escalation (default: sudo). become_user: specifies which user to escalate to (default: root). become_method: sudo (default), su, pbrun, pfexec, runas (Windows). Can be set at play level, task level, or in ansible.cfg. Requires target user to have sudo access (configure in /etc/sudoers). Use become_flags: '-s' to get a login shell. Avoid become at play level if only some tasks need it — use at task level for minimal privilege.

★★★★■ Frequently Asked Questions

★★★★■ How do you pass variables to a playbook at runtime?

Three ways: (1) -e 'key=value' or -e '{"key":"val"}' or -e @file.yml on command line — these have HIGHEST precedence (22). (2) --extra-vars same as -e. (3) vars_prompt: in play to prompt user interactively (useful for passwords in manual runs). In CI/CD: always use -e or vars files, never interactive prompts.

★★★★■ What is the 'check mode' and 'diff mode'?

--check (dry run): simulates playbook execution without making changes. Modules report what WOULD change. Not all modules support check mode. --diff: shows line-by-line differences for file changes (like git diff). Combine: ansible-playbook --check --diff — perfect for reviewing changes before applying. Add check_mode: true to individual tasks to always run in check mode.

★★★★■ Explain Jinja2 templating in Ansible — common filters.

Jinja2 is Ansible's templating engine. {{ var }} for variable substitution. {% %} for control structures (if, for, set). {# #} for comments. Key filters: default(), int/float/bool/string (type conversion), upper/lower/title, join, length, sort, unique, select/reject/selectattr, ternary, password_hash (for user passwords), b64encode/b64decode, to_json/from_json. Filters chain: {{ mylist | sort | unique | join(', ') }}

★★★★■ What is the difference between vars/ and defaults/ in a role?

defaults/main.yml: LOWEST priority variables — designed to be overridden by users of the role. This is where role authors put sensible defaults (nginx_port: 80). vars/main.yml: HIGHER priority — hard to override without extra_vars. Use vars/ for role-internal constants that shouldn't be changed. Best practice: put everything in defaults/ unless the variable must not be changed.

★★★★■ How do you test Ansible roles? What tools do you use?

Molecule: the standard testing framework for Ansible roles. Workflow: molecule create → runs tasks in Docker/Vagrant → molecule verify (Testinfra/Ansible asserts) → molecule destroy. ansible-lint: static analysis — checks for best practices and common errors. yamllint: YAML syntax checking. Testinfra (Python) or Goss: verify server state after role application. In CI: run molecule test on every PR.

★★★★■ What is 'gather_subset' and when would you use it?

gather_subset controls which facts are collected. Default collects everything. gather_subset: ['min'] — hostname, OS, python version only (very fast). gather_subset: ['!all', '!any', 'network'] — only network facts. gather_subset: ['hardware'] — CPU, RAM details. Use to speed up playbooks that only need specific facts, or on large fleets where full fact gathering is slow.

★★★★■ How do you use 'delegate_to' and what are common use cases?

delegate_to: runs a task on a DIFFERENT host than the current target. delegate_to: localhost — run on control node (AWS API calls, local file ops). delegate_to: bastion — run through bastion host. Common uses: deregister from LB before deploy, notify monitoring system, run DB migrations from a specific host, take snapshots before upgrade. run_once: true with delegate_to: runs only once regardless of how many hosts in play.

★★★★■ What is 'until' and how do you implement retry loops?

until: retries a task until condition is true. Requires retries: and delay:. Example: wait for service to be healthy after deploy. until: result.status == 200 retries: 10 delay: 5 Will try up to 10 times, waiting 5 seconds between attempts. If condition never true after retries, task fails. Use with uri: for health checks, with command: for process checks, wait_for_connection: for SSH ready.

★★★★■ How do you handle different Linux distributions in one playbook?

Use when: with ansible_os_family (RedHat, Debian) or ansible_distribution (CentOS, Ubuntu, Amazon). Pattern 1: separate task files — include_tasks: '{{ ansible_os_family }}.yml'. Pattern 2: vars_files by OS — vars/RedHat.yml, vars/Debian.yml. Pattern 3: use package: (OS-agnostic) instead of yum:/apt: where possible. Pattern 4: group_by: module to create dynamic groups by OS family.

★★★★■ Explain Ansible's connection plugins — SSH, WinRM, local.

Connection plugins define HOW Ansible connects to managed nodes. ssh (default): OpenSSH for Linux/Unix — uses SSH keys or passwords. local: run on control node itself — no remote connection. winrm: Windows Remote Management for Windows hosts. docker: connect to Docker containers. aws_ssm: AWS Systems Manager Session Manager — no SSH port needed. Set in ansible.cfg (default) or per-host: ansible_connection=winrm.

★★★★■ What is the 'meta' module used for?

meta: tasks control playbook execution behavior. meta: flush_handlers — run notified handlers immediately, not at end of play. meta: clear_facts — reset gathered facts. meta: end_play — stop executing remaining tasks in current play. meta: end_host — stop executing tasks for current host only. meta: reset_connection — re-establish SSH connection (useful after reboot tasks). meta: refresh_inventory — reload dynamic inventory mid-play.

★★★★■ How do you manage different environments (dev/staging/prod)?

Use separate inventory directories: inventories/dev/, inventories/staging/, inventories/prod/. Each has: hosts (or hosts.yml) + group_vars/ + host_vars/ directories. Shared vars in group_vars/all/, environment-specific in group_vars/env_name/. Separate vault files per environment with different vault passwords. Run: ansible-playbook -i inventories/prod/ site.yml --vault-id prod@~/.vault_pass_prod Never mix environment inventories. Never share vault passwords between envs.

★★★■■ Advanced / Tricky Questions (Senior SRE Level)

★★★■■ What are lookup plugins? Give examples.

Lookup plugins retrieve data from external sources and return it as a variable. They run on the CONTROL node (not managed node). {{ lookup('file', '/etc/hosts') }} — read local file. {{ lookup('env', 'HOME') }} — read environment variable. {{ lookup('password', '/tmp/pass length=16') }} — generate random password. {{ lookup('url', 'https://api.github.com/repos/ansible/ansible') }} — HTTP GET. {{ lookup('amazon.aws.aws_secret', 'prod/db_pass') }} — AWS Secrets Manager. {{ lookup('pipe', 'date') }} — run local command and capture output. Lookups are evaluated at runtime and not cached by default.

★★★■■ Explain the difference between 'notify' and 'listen' in handlers.

notify: specifies the handler name to trigger. The handler name must match EXACTLY. listen: allows a handler to subscribe to a topic name, not just its own name. Multiple handlers can listen: to the same topic. Multiple tasks can notify: the same topic. This decouples tasks from specific handler names — useful in roles where the handler name might be defined in the consuming playbook, not the role. Example: notify: restart services → handler: listen: restart services

★★★■■ What is 'strategy: free' and when would you use it?

Default strategy is 'linear': all hosts must complete each task before Ansible moves to next task. With strategy: free, each host races ahead independently — fast hosts don't wait for slow ones. Use for: long-running tasks where host speed varies significantly, batch jobs, data processing across many heterogeneous servers. Downside: harder to reason about execution order, handlers may trigger at unexpected times. host_pinned: like free but pins one worker thread per host — good for connection limits.

★★★■■ How do you create a custom Ansible module?

Custom modules are Python scripts that accept JSON input and return JSON output. Place in library/ directory (role or playbook level). Import AnsibleModule from ansible.module_utils.basic. Define argument_spec for input parameters. Call module.exit_json(changed=True/False, result={}) on success. Call module.fail_json(msg='error') on failure. Ansible handles check_mode, diff, become automatically via AnsibleModule class. For simple wrappers, use action plugins; for new functionality, use full modules.

★★★■■ What is 'run_once' and what pitfalls does it have?

run_once: true causes a task to execute ONCE across all hosts in the current play batch. Useful for: DB migrations, sending one notification, creating a lock file. It runs on the FIRST host in the play (alphabetically, or the first active). Pitfall: if using serial:, run_once runs once PER BATCH, not once total! Combine with delegate_to: localhost for truly single execution. Variables registered with run_once are only available on the host it ran on — access from other hosts via hostvars[first_host].variable_name.

★★★■■ How does Ansible handle Windows hosts differently?

Connection: WinRM instead of SSH (ansible_connection: winrm). Requires: pywinrm Python package on control node. Win modules: win_command, win_shell, win_copy, win_template, win_service, win_package, etc. Setup: Enable WinRM via PowerShell script (ConfigureRemotingForAnsible.ps1). Auth: Basic, Kerberos (domain joined), NTLM, Certificate. No become/sudo — use runas instead (ansible_become_method: runas). Paths use Windows format in win_ modules. Alternative: SSH on Windows 2019+ (OpenSSH server built-in).

★★★■■ What is the 'serial' batching with pre/post tasks pattern for zero-downtime deploys?

Pattern: serial: '25%' ensures only 25% of fleet is updated at once. pre_tasks: remove host from load balancer (delegate_to: lb_host). tasks/roles: stop service, deploy new code, start service. post_tasks: health check → re-add to load balancer. If health check fails: rescue block rolls back, host stays out of rotation. Handlers: restart service (only if changed). max_fail_percentage: 10 aborts if >10% fail. This ensures at minimum 75% of fleet is always serving traffic.

★★★■■ How do you use 'ansible-pull' and when is it appropriate?

ansible-pull inverts the model: managed nodes PULL and run playbooks from a git repo. Each node runs ansible-pull via cron/systemd timer. Use cases: massive scale (10,000+ nodes where push doesn't scale), nodes in isolated networks that can reach git but not be reached by control node, self-healing infrastructure (nodes continuously enforce their own config). Tradeoff: harder to orchestrate multi-host operations, results not centrally visible (need callback plugins or AnsibleTower/AWX). Push still preferred for most SRE use cases; pull for large scale or air-gapped nodes.

★★★■■ What are callback plugins? Name useful ones for production.

Callback plugins hook into Ansible events (task start, result, playbook end) to extend behavior. stdout_callback: determines terminal output format. yaml: pretty YAML output (cleaner than default). profile_tasks: shows timing for each task — find slow tasks. profile_roles: timing per role. mail: send email on failure. slack: send Slack notification on play completion. splunk, datadog, grafana: send metrics to monitoring platforms. junit: generate JUnit XML for CI systems. Set in ansible.cfg: [defaults] callback_whitelist = profile_tasks,slack

★★★■■ How do you implement idempotent shell commands?

Option 1: creates: / removes: args — skip if file exists/absent. Option 2: changed_when: false — always 'ok', never 'changed'. Option 3: changed_when: result.stdout != '' — custom changed condition. Option 4: register result + when condition on next task. Option 5: stat: module to check state before shell command. Best practice: prefer Ansible modules over shell — they're inherently idempotent. Use shell only when no module exists. Always add changed_when and failed_when.

★★★■■ Explain Ansible Tower / AWX — what problems does it solve?

Ansible Tower (commercial) / AWX (open-source) is a web UI and API layer over Ansible. Problems solved: (1) Centralized execution — no need for everyone to have SSH keys. (2) RBAC — control who can run which playbooks on which hosts. (3) Audit trail — complete log of every execution. (4) Scheduled jobs — cron-like execution without cron complexity. (5) Inventory sync — auto-sync dynamic inventory from cloud. (6) Secrets management — credential store with no plain-text secrets. (7) Workflow templates — chain multiple playbooks with conditions. For teams >3-4 people running Ansible in production, Tower/AWX is essentially required.

★★★■■ How do you handle secrets in CI/CD pipelines with Ansible?

Never put vault passwords in code. Store in CI secret store. GitHub Actions: secrets.ANSIBLE_VAULT_PASSWORD → write to temp file in job. Jenkins: credentials plugin → inject as environment variable. GitLab CI: CI/CD variables (masked) → available as $VAULT_PASSWORD. Pattern: echo '$ANSIBLE_VAULT_PASSWORD' > ~/.vault_pass && chmod 600 ~/.vault_pass Cleanup: always delete in after_script/post-step. Alternative: use AWS Secrets Manager lookup — IAM role in CI runner grants access, no vault password needed at all. Best for AWS-native pipelines.

■ Tricky Questions — What Interviewers Really Want to Know

■ TRICKY: If gather_facts: false, can you still use ansible_hostname?

NO — ansible_hostname is a fact gathered by the setup module. If gather_facts: false, it will be undefined. However, inventory_hostname is always available — it's a magic variable, not a fact. Workaround: add a task 'setup: gather_subset: [min]' to get minimal facts including hostname. This is a common gotcha when optimizing for speed.

■ TRICKY: Can you have the same handler name in a role and a play? What happens?

Yes — but scope matters. A handler in a role is scoped to that role's context. A handler in the play-level handlers: section is play-scoped. If a role task notifies 'restart nginx', it finds the handler within the role first. If not found in role, searches play-level handlers. Naming collision can cause unexpected behavior — use unique names or use listen: topics.

■ TRICKY: Why does 'command: cd /tmp && ls' fail but 'shell: cd /tmp && ls' works?

command: does not invoke /bin/sh — it runs the command directly without shell interpretation. The && operator and cd are shell features. command: cd /tmp && ls tries to run an executable called 'cd' with args '&&' and 'ls'. shell: wraps the command in /bin/sh -c '...' so shell features work. Always use shell: when you need pipes (|), redirects (>, >>), or operators (&&, ||).

■ TRICKY: What happens if you use loop: with import_tasks:?

It fails at parse time — import_tasks is STATIC and processed before runtime. Variables from loop: are only available at runtime. include_tasks: is DYNAMIC — it works with loop: because it's processed at runtime. This is the most common import vs include gotcha. Rule: if you need to loop over a task file, always use include_tasks:.

■ TRICKY: If set_fact stores a variable, is it available to other hosts?

No — set_fact stores variables PER HOST in the current play. It's available for subsequent tasks on THAT HOST only. To share a fact from host A to host B: hostvars['host-a']['my_fact'] — access via hostvars magic variable. For this to work, host A must have already run the set_fact task (plays run on all hosts sequentially).

■ TRICKY: Can Ansible's 'when' condition reference a variable that doesn't exist?

Yes — but carefully. Undefined variables in when: conditions raise an error by default. Safe pattern: when: myvar is defined and myvar == 'value' Or use default filter: when: myvar | default('') == 'value' Using when: myvar == 'value' with undefined myvar will fail. Use is defined / is undefined tests to guard against this.

■ TRICKY: What's the difference between 'any_errors_fatal' and 'max_fail_percentage: 0'?

any_errors_fatal: true — if ANY host fails, immediately abort the entire play. No more tasks on any host. max_fail_percentage: 0 — abort if MORE THAN 0% fail, but still finish the current task on other hosts. any_errors_fatal stops everything immediately. max_fail_percentage: 0 is checked between serial batches — current batch finishes. Use any_errors_fatal for critical pre-checks. Use max_fail_percentage for deployment tolerance.

■ TRICKY: How do Ansible's 'until' loops interact with 'ignore_errors'?

ignore_errors: true on an until: task — if retries are exhausted and condition never true, the task fails but Ansible continues. The registered result will have failed: true. Without ignore_errors, exhausted retries halt the play. Common pattern: run health check with until:, use ignore_errors: to capture failure gracefully, then check result.failed in a subsequent when: condition to take specific action.

MODULE 11 — ANSIBLE CHEAT SHEET — Quick Revision Reference

Essential Commands

```
# Essential Ansible commands & ansible.cfg
# ■■ Ad-hoc commands ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ansible all -m ping                                  # Test connectivity
ansible webservers -m setup                          # Gather facts
ansible all -m shell -a 'uptime' --one-line          # Quick status
ansible all -m package -a 'name=nginx state=present' -b  # Install package
ansible all -m service -a 'name=nginx state=restarted' -b

# ■■ Playbook execution ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ansible-playbook site.yml                            # Basic run
ansible-playbook site.yml -i inventories/prod/       # Specific inventory
ansible-playbook site.yml --limit webservers         # Limit hosts
ansible-playbook site.yml --tags deploy              # Only tagged tasks
ansible-playbook site.yml --skip-tags packages       # Skip tags
ansible-playbook site.yml -e 'version=2.1.0'         # Extra vars
ansible-playbook site.yml --check --diff             # Dry run + diff
ansible-playbook site.yml --start-at-task 'Deploy app'
ansible-playbook site.yml -v / -vv / -vvv / -vvvv    # Verbosity

# ■■ Inventory ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■
ansible-inventory -i inventory/ --list               # List all hosts+vars JSON
ansible-inventory -i inventory/ --graph              # Tree view
ansible-inventory -i aws_ec2.yml --list              # Dynamic inventory

# ■■ Galaxy ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■
ansible-galaxy install -r requirements.yml
ansible-galaxy collection install amazon.aws
ansible-galaxy role init myrole                      # Scaffold new role
ansible-galaxy collection init myorg.mycollection

# ■■ Vault ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■
ansible-vault create secrets.yml
ansible-vault edit secrets.yml
ansible-vault encrypt_string 'value' --name 'varname'
ansible-vault rekey secrets.yml

# ■■ ansible.cfg — key settings ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
[defaults]
inventory = ./inventories
remote_user = ec2-user
host_key_checking = False      # Dev only!
forks = 50
gathering = smart
fact_caching = jsonfile
fact_caching_connection = /tmp/ansible_facts
fact_caching_timeout = 86400
vault_password_file = ~/.vault_pass
roles_path = ./roles:~/.ansible/roles
```

```
stdout_callback = yaml
callback_whitelist = profile_tasks

[ssh_connection]
pipelining = True
ssh_args = -o ControlMaster=auto -o ControlPersist=60s
```

Key Concepts Summary

Concept	Remember This
Agentless	SSH (Linux) / WinRM (Windows). Only Python required on managed nodes.
Idempotency	Modules enforce desired state — safe to re-run. shell/command need changed_when:.
Variable Precedence	Extra vars (-e) > set_fact > vars/ > group_vars > defaults/. Know the 22 levels.
Roles vs Collections	Role = one function. Collection = package of roles+modules+plugins.
defaults/ vs vars/	defaults/ = lowest priority (meant to override). vars/ = high priority (constants).
import vs include	import = static (parse time, tags inherit). include = dynamic (runtime, loop works).
Handlers	Run ONCE at play end, only if notified AND task changed. flush_handlers for mid-play.
serial:	Rolling updates. [1, 5, 100%] = progressive. max_fail_percentage for safety.
become:	sudo/su privilege escalation. Per-task is better than per-play (least privilege).
Vault	AES-256. Encrypt whole files or inline strings. Use vault_ prefix convention.
Dynamic Inventory	Cloud-native host discovery. aws_ec2 plugin for AWS. No manual host management.
check + diff mode	--check --diff = dry run with file diff. Essential before prod changes.
Pipelining	pipelining=True in ansible.cfg. Biggest single performance boost. Disable requiretty.
gather_facts: false	Speed optimization. Use gather_subset: [min] for minimal facts.
delegate_to:	Run task on different host. delegate_to: localhost for control node tasks.
run_once:	Per batch, not per play! Combine with delegate_to: localhost for truly once.
Strategy: free	Hosts race ahead independently. Better for heterogeneous, long-running tasks.
Mitogen	Third-party strategy plugin. 3-10x faster. Install separately.
AWX/Tower	Web UI + RBAC + audit + scheduling + secrets. Required for team Ansible usage.

Interview Priority Matrix

Must Know ★★★★★	Know Well ★★★★■	Know Basics ★★★■■
Agentless architecture	Dynamic inventory (AWS)	Callback plugins
Variable precedence (22 levels)	Molecule testing	Custom modules
Idempotency + changed_when	Mitogen strategy	ansible-pull

Roles structure + defaults vs vars	AWX/Tower capabilities	action plugins
import vs include	Windows (WinRM)	lookup plugins (all)
Handlers behavior	Serial + max_fail_%	strategy: free internals
Ansible Vault	Pipelining + forks tuning	until + ignore_errors combo
Dynamic inventory concept	delegate_to patterns	meta: module
become + privilege escalation	CI/CD pipeline integration	gather_subset details
Jinja2 filters (key ones)	Multi-environment inventory	run_once pitfalls
command vs shell vs raw	Galaxy requirements.yml	Collections authoring

■ SRE Interview Winning Framework:
When answering Ansible questions, follow: **What** → **Why** → **How** → **Production Example** → **Trade-offs**

Example: 'What is serial:?' → Define it (batch size for rolling updates) → Why (limit blast radius) → How (serial: '25%' in play) → Production: used for zero-downtime nginx upgrades across 200 servers → Trade-off: slower deployment but safer. Each batch is fully healthy before next starts.

Designed for Cloud Operations Engineers → Senior SRE · DevOps Engineer · Platform Engineer · Infrastructure Engineer